Formulas

Triangular numbers: $\sum_{i=0}^{n} i = \frac{n(n-1)}{2}$

Geometric series: $\sum_{i=0}^{n} x^i = \frac{x^{n+1}-1}{x-1}$

Binomial coefficients: $\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$

Log properties: $\log(c^a) = a \log c$, $\log(ab) = \log a + \log b$, $\log_b(b) = 1$

Expectation: $E[X] = \sum_i i * \Pr[X = i]$

Linearity of expectation: $E[a + b] = E[a] + E[b]$

Big-O ranking of functions (here $f(n) < g(n)$ means "$f(n)$ is $O(g(n))$"): $\log n < \sqrt{n} < n < n^k < 2^n < n!$

Runtimes

In all of these, n is the size of the list, heap, BST, or array. All times are worst-case. h is the height of the BST, which is $O(\log n)$ if using a self-balancing BST such as an AVL tree.

Heaps	
swap-up and swap-down	$O(\log n)$
max	$O(1)$
build heap	$O(n \log n)$
build heap from array	$O(n)$
BSTs	
build bst	$O(n \log n)$
build bst from sorted array	$O(n)$
tree traversals (pre-order, in-order, post-order, level-order)	$O(n)$
search, insert, delete	$O(h)$
successor, predecessor	$O(h)$
Arrays and lists	
sort a list	$O(n \log n)$
merge two sorted lists (m is the size of the second list)	$O(n + m)$
insert in array	$O(n)^*$
insert in linked list	$O(1)^*$
access array item at index	$O(1)$
access linked list item at index	$O(n)$
Hash tables	
add x to hash table	$O(n)^*$
look up x in hash table	$O(n)^*$
delete x from hash table	$O(n)^*$

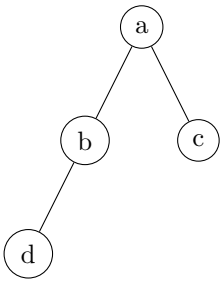
Inserting in an array is $O(1)$ unless the array is full, and is $O(1)$ amortized using dynamically sized arrays.

Inserting in a linked list is $O(1)$ plus the time taken to find where to insert.

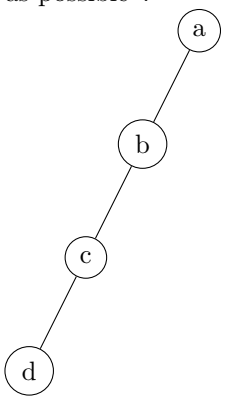
Hash tables are $O(1)$ average, assuming the hash function is good and the load factor is small.

Nearly complete

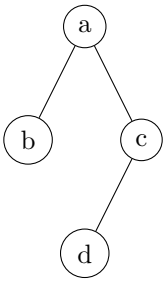
An n-node tree is nearly complete if all leaf nodes are at height h or $h-1$, and all leaves are "as far left as possible".



Nearly complete



Not nearly-complete because the tree height is $> \lceil \log 4 \rceil$



Not nearly-complete because a node with children (c) is right of a node without children (b)

Balance Factor

The balance factor of a node in a binary tree is the height of its right subtree minus the height of its left subtree.

A tree is AVL-balanced if all nodes have balance factor -1, 0, or +1.